# Enzyme Discovery Agent

Finding enzymes for a reaction —
without ever telling the agent the answer

the model picks the strategy

the tools find the enzymes

the answer stays hidden

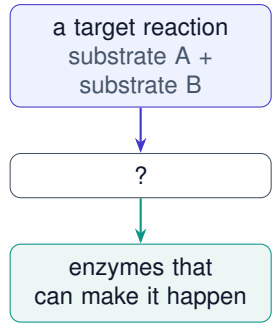
# The problem

You give it a reaction — some substrates turning into a product — and it has to find enzymes that could catalyze it.

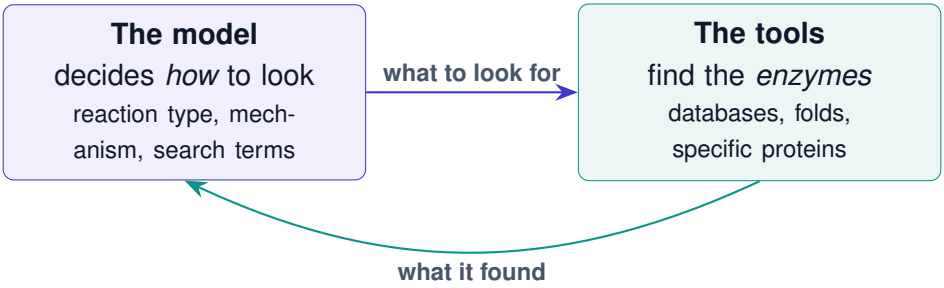
That's hard because:

- the reaction is often something **nature doesn't do**, so nothing is labelled for it
- there's nothing to look up — the answer has to be **discovered**
- and the haystack is **hundreds of millions** of proteins

The catch: hand the agent the answer's family up front, and it isn't really **discovering** anything.



# The idea



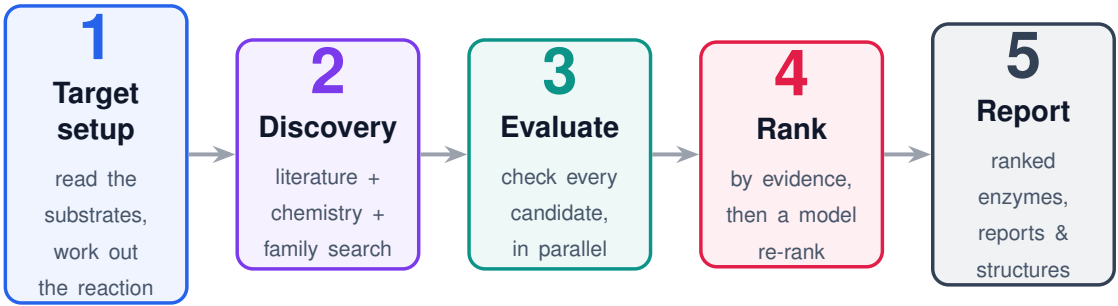

**We keep the answer out of the agent**:

- its enzymes, substrates, and family never appear in a prompt, a tool, or even a comment
- the real answer lives **only** in a test the agent never sees

**We let the evidence do the deciding**:

- the code measures the facts — direction, how well a substrate fits — and the model reasons over them
- no hand-tuned cutoffs; things are ranked against each other, not

# How it works, end to end




The gates make sure the important steps actually happen, while the model still decides what to chase inside the loop.

# Three ways it finds candidates

**From the literature**

Read a few open papers and pull out the enzymes they name or cite. Often the only way to reach something that lives in a recent paper — it's how we got into the answer's **own gene cluster**.

**From the chemistry**

The model lists the different **folds and chemistries** that make this kind of bond, and we drop any that run the **wrong way** — a bond-breaking fold can't build one.

**From the fold**

For each family, look at its members, see whose natural substrate looks most **like our target**, and add the good ones. This is the main engine.

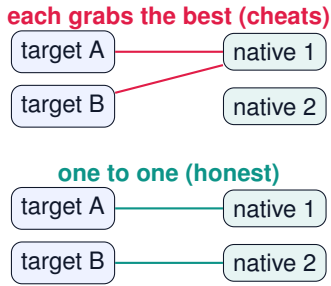
One thing we learned: judge substrate fit **member by member**, not by the family's textbook example — the two are rarely the same.

# Making the substrate match honest

When the target has two substrates, we don't want it to **cheat**. If each one just grabs the enzyme's single best native, both can point at the *same* one — and a bad second partner slips by unnoticed.

So we match them **one to one**: each native can serve only one target substrate, and we score the enzyme by its **worst** pairing.

Now the target's amine is forced onto the enzyme's amine — and a tiny donor like ammonia finally scores as the poor match it is.



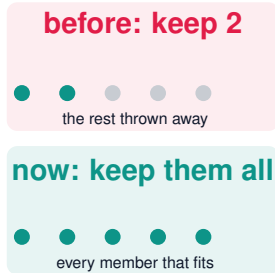
# The whole point is not to miss anything

## Keep every member that fits

We used to keep only a family's best two. Now every member that runs the right direction and has a real substrate to compare becomes a candidate, so a big family's long tail doesn't get thrown away.

## And actually evaluate all of them

Evaluation is just database lookups, no model calls, so looking at more of them is basically free. We check every distinct gene at once — so a budget never quietly drops one that mattered.




It costs a bit more, but finding everything matters more.

# How the final list is ordered

- A **fixed, reproducible core** sorts candidates by hard evidence — the bond that changes, the direction, how well the substrate fits. No magic weights.
- If the agent decided a family was **off-target**, its members sink to the bottom; members of on-target families float up.
- **Only then** does the model re-rank the short list at the top — a thin, checkable layer over an order we can already trust.

direction check → worst-substrate fit → family relevance → model re-rank

# Does it work?

Our test case: find enzymes that join a six-carbon diacid and a six-carbon diamine — the building blocks of nylon-6,6. The correct answers sit in a separate test the agent never sees; we only use them to grade the run afterwards.

**What comes back**

- the top hits are genuine **look-alikes** — enzymes that join citrate to a diamine, chemically close to what we want
- the exact answer, **DdaG**, shows up at #4 — found through the literature, sitting in its own gene cluster
- the family list is **clean**: no bond-breaking folds, and the old habit of over-ranking a poor match is gone

**And how it behaves**

- a run dropped from **53 to 32 minutes** once we trimmed the reading
- almost all of that time is the **model thinking**; the tools take about 2%
- every candidate we screen now actually gets evaluated

# What changed recently

## Finding better enzymes

- match substrates one to one, the same way everywhere
- keep — and evaluate — every enzyme that fits
- make the family list respect reaction direction

## Making it lighter

- much less literature reading, and it no longer stalls on paywalls
- evaluate candidates in parallel

## Cleaning up the code

- folded nine gates into one clean pipeline
- removed duplicated scoring and direction logic
- deleted about 730 lines that weren't doing anything

**all tests pass** **answer still hidden**

# What's still rough

- **Literature.** We still miss some of the best papers because we search with the wrong words — and a few are behind paywalls. Rewriting the queries helps, but not reliably yet.
- **Speed.** Nearly all the time is the model reading long prompts. The wins are shorter prompts and doing the final steps in parallel.
- **Ranking.** A couple of scoring formulas still don't quite agree, and we want a full run to confirm the one-to-one change helps in practice.
- **Coverage.** Make sure we really look at every member of a family, not just the ones we happen to reach.

Everything ships with the tests passing, the answer still hidden, and a note in the log.

# Discover it, don’t look it up.

The model decides how to search, the tools find the enzymes, and the answer never gets to peek at itself.

**honest substrate matching** **keep everything that fits** **answer stays hidden**